

**UNIVERSIDAD PERUANA LOS ANDES
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE
SISTEMAS Y COMPUTACIÓN**



CURSO: Base de Datos II

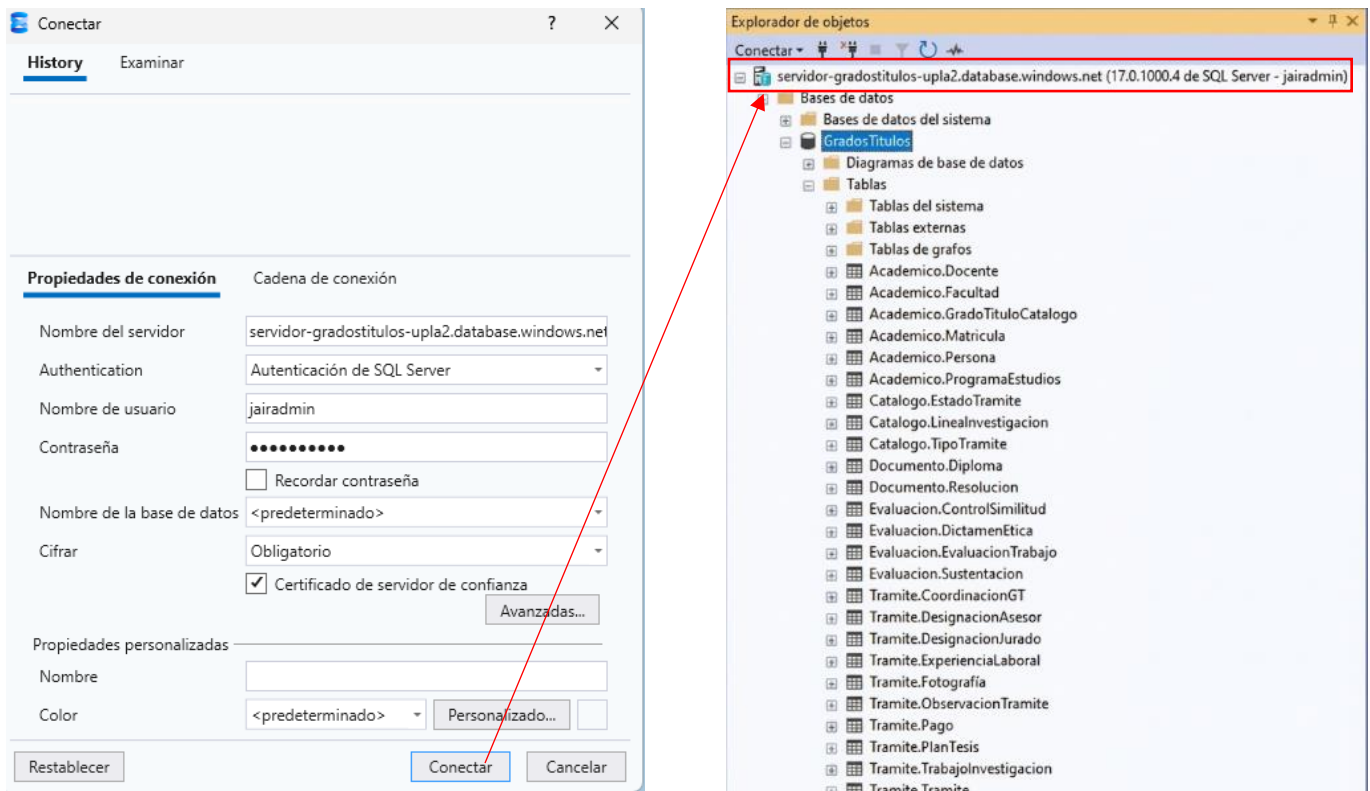
DOCENTE: Ing. Fernandez Bejarano Raul Enrique

ESTUDIANTE: Nuñez Michel Jair

CICLO: V

SECCION: B1

Conectar Azure SQL en SQL Server Management Studio 22 y desarrollo de todos los enunciados y ejercicios:



Proyecto 1: Creación y distribución de archivos físicos de la base GradosTitulos

1. Enunciado: Crea la base de datos GradosTitulos con un archivo primario, un archivo secundario de datos para optimizar las consultas de documentos históricos, y el log de transacciones en rutas físicas dedicadas dentro del servidor. Posteriormente, realiza la consulta de los archivos físicos asociados para validar la distribución.
2. Script de la solución en T-SQL

```
-- =====
-- 1. CREAR BASE DE DATOS
-- =====
IF EXISTS (SELECT name FROM sys.databases WHERE name = N'GradosTitulos')
BEGIN
    ALTER DATABASE GradosTitulos SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE GradosTitulos;
END
GO

-- Creación física distribuida (incluye grupo secundario para Documentos)
CREATE DATABASE GradosTitulos
ON PRIMARY (
    NAME = GradosTitulos_Data,
    FILENAME = 'C:\BaseDatos2026\GradosTitulos.mdf',
    SIZE = 20MB,
    MAXSIZE = 20GB,
    FILEGROWTH = 5MB
),
FILEGROUP DocumentosGroup (
    NAME = GradosTitulos_DocData,
    FILENAME = 'C:\BaseDatos2026\GradosTitulos.ndf',
    SIZE = 10MB,
    MAXSIZE = 10GB,
    FILEGROWTH = 2MB
)
LOG ON (
    NAME = GradosTitulos_Log,
    FILENAME = 'C:\BaseDatos2026\GradosTitulos.ldf',
    SIZE = 6MB,
    MAXSIZE = 2GB,
    FILEGROWTH = 2MB
);
GO

SELECT
    df.name AS NombreLogico,
    df.physical_name AS RutaFisica,
    df.type_desc AS TipoArchivo,
    fg.name AS GrupoArchivos,
    (df.size * 8) / 1024 AS TamanoActual_MB
FROM sys.database_files df
LEFT JOIN sys.filegroups fg ON df.data_space_id = fg.data_space_id;
GO
```

100 % No se encontraron problemas.						Línea: 34 Carácter: 1 SPC CRLF		
Resultados Mensajes								
	NombreLogico	Logico	RutaFisica	TipoArchivo	GrupoArchivo	TamanoActual_MB		
1	data_0		a3be0779-926a4db1-8bc1-03dbfa38e97.mdf	ROWS	PRIMARY	32		
2	log		a3be0779-926a4db1-8bc1-03dbfa38e927.ldf	LOG	NULL	8		
3	XTP		a3be0779-926a4db1-8bc1-03dbfa38e927.xtp	FILESTREAM	NULL	0		

Consulta ejecutada correct correctamente. servidor-gradostitulos-upla2... jairadmin(72) GradosTitulos 00:00:00 3 filas

- Justificación técnica de la solución aplicada La separación de archivos en un grupo primario (.mdf) y secundario (.ndf) permite aislar tablas operativas de alta transaccionalidad de aquellas que almacenan datos históricos pesados (como la tabla Documento.Diploma o Tramite.Fotografia). Esto disminuye la contención de I/O en los discos del servidor, optimizando los tiempos de respuesta de las cabezas de lectura/escritura y flexibilizando las estrategias de respaldo parcial.
- Explicación de las buenas prácticas utilizadas en el proyecto Se implementa la cláusula SET SINGLE_USER WITH ROLLBACK IMMEDIATE antes del descarte de la base de datos para asegurar que ninguna conexión persistente bloquee la recreación del entorno. Además, se definieron parámetros controlados de crecimiento (FILEGROWTH), evitando el uso de valores por defecto o porcentuales que degradan el rendimiento por fragmentación física

Ejercicio Propuesto 1: Incorporación de un grupo de archivos exclusivo para Índices Enunciado

Modifica la estructura de la base de datos **GradosTitulos** agregando un nuevo grupo de archivos llamado **IdxGroup** con un archivo físico secundario (.ndf). Esto servirá para almacenar de forma aislada todos los índices no agrupados de la base de datos (como **IX_Persona_DNI** o **IX_Tramite_Matricula**), optimizando la velocidad de las búsquedas.

Script de la solución en T-SQL

```
USE master;
GO

-- Agregar un nuevo grupo de archivos para almacenar índices
ALTER DATABASE GradosTitulos
ADD FILEGROUP IdxGroup;
GO

-- Agregar un archivo físico secundario al nuevo grupo de archivos
ALTER DATABASE GradosTitulos
ADD FILE
(
    NAME = GradosTitulos_IdxData,
    FILENAME = 'C:\BaseDatos2026\GradosTitulos_idxdata.ndf',
    SIZE = 10MB,
    MAXSIZE = 5GB,
    FILEGROWTH = 5MB
)
TO FILEGROUP IdxGroup;
GO

-- Consulta de verificación del nuevo grupo de archivos
USE GradosTitulos;
GO

SELECT
    df.name AS NombreLogico,
    df.physical_name AS RutaFisica,
    df.type_desc AS TipoArchivo,
    fg.name AS GrupoArchivos,
    (df.size * 8) / 1024 AS TamanoActual_MB
FROM sys.database_files df
LEFT JOIN sys.filegroups fg
ON df.data_space_id = fg.data_space_id
WHERE fg.name = 'IdxGroup';
GO
```

Justificación técnica de la solución aplicada

La incorporación del grupo de archivos **IdxGroup** permite separar los índices del almacenamiento principal de las tablas, mejorando la organización de la base de datos y optimizando las operaciones de búsqueda.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó **ALTER DATABASE** para modificar la estructura de la base de datos sin recrearla. Además, se definieron parámetros de crecimiento del archivo y se verificó correctamente la creación del nuevo grupo de archivos.

Ejercicio Propuesto 2: Reubicación de un Filegroup y consulta de espacio disponible

Enunciado

Desarrolla una consulta transaccional avanzada que muestre el espacio total asignado, el espacio utilizado y el espacio libre (en megabytes) para cada uno de los grupos de archivos físicos que componen actualmente la base de datos **GradosTitulos**.

Script de la solución en T-SQL

```
-- Consulta del espacio asignado, utilizado y libre por cada grupo de archivos

SELECT
    fg.name AS GrupoArchivos,
    df.name AS NombreLogico,
    df.physical_name AS RutaFisica,
    CAST((df.size * 8.0) / 1024 AS DECIMAL(10,2)) AS EspacioTotal_MB,
    CAST((FILEPROPERTY(df.name, 'SpaceUsed') * 8.0) / 1024 AS DECIMAL(10,2)) AS EspacioUtilizado_MB,
    CAST(((df.size - FILEPROPERTY(df.name, 'SpaceUsed')) * 8.0) / 1024 AS DECIMAL(10,2)) AS EspacioLibre_MB
FROM sys.database_files df
LEFT JOIN sys.filegroups fg
    ON df.data_space_id = fg.data_space_id
ORDER BY fg.name, df.name;
GO
```

	GrupoArchivos	NombreLogico	EspacioTotal_MB	EspacioUtilizado_MB	EspacioLibre_MB
1	NULL	GradosTitulos_Log	6.00	0.41	5.59
2	DocumentosGroup	GradosTitulos_DocData	10.00	0.06	9.94
3	IdxGroup	GradosTitulos_IdxData	10.00	0.06	9.94
4	PRIMARY	GradosTitulos_Data	20.00	5.13	14.88

Justificación técnica de la solución aplicada

La consulta permite conocer el espacio total, utilizado y libre de cada grupo de archivos de la base de datos, facilitando el monitoreo del almacenamiento y la planificación del crecimiento de los archivos físicos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizaron las vistas de catálogo del sistema y la función **FILEPROPERTY** para obtener información confiable del almacenamiento. Además, los resultados se presentan en megabytes para facilitar su interpretación.

Proyecto 2: Ajuste de configuración y validación de propiedades de GradosTitulos

Enunciado

Consulta las propiedades de colación actuales de la base de datos. Modifica la colación de **GradosTitulos** para asegurar el soporte nativo de caracteres en español (tildes y la letra 'Ñ'). Adicionalmente, ajusta el crecimiento automático del archivo primario de datos de manera fija a **20 MB**.

Script de la solución en T-SQL

```

USE master;
GO

-- Consulta de la colación actual de la base de datos
SELECT
    name AS BaseDatos,
    collation_name AS ColacionActual
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Modificar la colación de la base de datos
ALTER DATABASE GradosTitulos
COLLATE Modern_Spanish_CI_AS;
GO

-- Configurar el crecimiento automático del archivo primario a 20 MB
ALTER DATABASE GradosTitulos
MODIFY FILE
(
    NAME = GradosTitulos_Data,
    FILEGROWTH = 20MB
);
GO

-- Verificar la nueva configuración
SELECT
    d.name AS BaseDatos,
    d.collation_name AS Colacion,|
    mf.name AS NombreArchivo,
    (mf.growth * 8) / 1024 AS Crecimiento_MB
FROM sys.databases d
INNER JOIN sys.master_files mf
ON d.database_id = mf.database_id
WHERE d.name = 'GradosTitulos'
AND mf.name = 'GradosTitulos_Data';
GO

```

Justificación técnica de la solución aplicada

La modificación de la colación permite el manejo adecuado de caracteres propios del idioma español, mientras que el crecimiento fijo de **20 MB** mejora el control del almacenamiento del archivo principal de la base de datos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se verificó la configuración antes y después de realizar los cambios. Además, se estableció un crecimiento fijo del archivo de datos para evitar expansiones descontroladas y facilitar la administración del almacenamiento.

Ejercicio Propuesto 1: Elevación del Nivel de Compatibilidad (Compatibility Level) Enunciado

Verifica el nivel de compatibilidad actual de **GradosTitulos** y elévalo a la última versión disponible del motor de base de datos para habilitar optimizaciones de consulta avanzadas, como el procesamiento de consultas inteligentes (*Intelligent Query Processing*).

Script de la solución en T-SQL

```

USE master;
GO

-- Consultar el nivel de compatibilidad actual
SELECT
    name AS BaseDatos,
    compatibility_level AS NivelCompatibilidad
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Elevar el nivel de compatibilidad
ALTER DATABASE GradosTitulos
SET COMPATIBILITY_LEVEL = 170;
GO

-- Verificar el nuevo nivel de compatibilidad
SELECT
    name AS BaseDatos,
    compatibility_level AS NivelCompatibilidad
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

```

Justificación técnica de la solución aplicada

La actualización del nivel de compatibilidad permite que la base de datos aproveche las mejoras y optimizaciones disponibles en la versión más reciente del motor de SQL Server.

Explicación de las buenas prácticas utilizadas en el proyecto

Se verificó el nivel de compatibilidad antes y después del cambio para confirmar que la configuración fue aplicada correctamente.

Ejercicio Propuesto 2: Activación de la propiedad PAGE_VERIFY CHECKSUM para protección contra corrupción

Enunciado

Configura la propiedad de verificación de páginas de la base de datos **GradosTitulos** en modo **CHECKSUM** para asegurar que el motor detecte cualquier corrupción de datos a nivel físico causada por fallas en el disco o en el sistema operativo de almacenamiento.

Script de la solución en T-SQL

```

USE master;
GO

-- Consultar la configuración actual de PAGE_VERIFY
SELECT
    name AS BaseDatos,
    page_verify_option_desc AS VerificacionPaginas
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Configurar PAGE_VERIFY en modo CHECKSUM
ALTER DATABASE GradosTitulos
SET PAGE_VERIFY CHECKSUM;
GO

-- Verificar la configuración aplicada
SELECT
    name AS BaseDatos,
    page_verify_option_desc AS VerificacionPaginas
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

```

Justificación técnica de la solución aplicada

La configuración **PAGE_VERIFY CHECKSUM** permite detectar posibles errores o corrupción en las páginas de datos, incrementando la confiabilidad de la base de datos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se verificó la configuración antes y después de realizar el cambio para asegurar que la propiedad **CHECKSUM** fue aplicada correctamente.

Proyecto 3: Definición de modelo de recuperación y respaldo para GradosTitulos Enunciado

Cambia de forma secuencial el modelo de recuperación de la base de datos **GradosTitulos** a **SIMPLE** y posteriormente a **BULK_LOGGED**. Explica el impacto de ambos entornos. Finalmente, cambia el modelo a **FULL** y ejecuta un respaldo completo (**BACKUP DATABASE**) hacia un directorio seguro de almacenamiento.

Script de la solución en T-SQL

```
-- Cambiar el modelo de recuperación a SIMPLE
ALTER DATABASE GradosTitulos
SET RECOVERY SIMPLE;
GO

-- Verificar el modelo de recuperación
SELECT
    name AS BaseDatos,
    recovery_model_desc AS ModeloRecuperacion
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Cambiar el modelo de recuperación a BULK_LOGGED
ALTER DATABASE GradosTitulos
SET RECOVERY BULK_LOGGED;
GO

-- Verificar el modelo de recuperación
SELECT
    name AS BaseDatos,
    recovery_model_desc AS ModeloRecuperacion
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Cambiar el modelo de recuperación a FULL
ALTER DATABASE GradosTitulos
SET RECOVERY FULL;
GO

-- Verificar el modelo de recuperación
SELECT
    name AS BaseDatos,
    recovery_model_desc AS ModeloRecuperacion
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Realizar un respaldo completo de la base de datos
BACKUP DATABASE GradosTitulos
TO DISK = 'D:\Backups\GradosTitulos_Full.bak'
WITH
    FORMAT,
    INIT,
    NAME = 'Respaldo Completo GradosTitulos',
    STATS = 10;
GO
```

Justificación técnica de la solución aplicada

El cambio entre los modelos de recuperación **SIMPLE**, **BULK_LOGGED** y **FULL** permite adaptar la estrategia de recuperación según las necesidades de la base de datos. Finalmente, el respaldo completo garantiza la disponibilidad de una copia para la recuperación de la información.

Explicación de las buenas prácticas utilizadas en el proyecto

Se verificó el modelo de recuperación después de cada cambio y se realizó un respaldo completo en una ubicación específica para asegurar la protección de los datos.

Ejercicio Propuesto 1: Respaldo diferencial de seguridad para optimización de tiempos Enunciado

Con el objetivo de optimizar el espacio en el almacenamiento y acelerar la ventana de mantenimiento diario, ejecuta un respaldo de tipo diferencial (**DIFFERENTIAL**) de la base de datos **GradosTitulos**, guardándolo en un archivo específico.

Script de la solución en T-SQL

```
USE master;
GO

-- Realizar un respaldo diferencial de la base de datos
BACKUP DATABASE GradosTitulos
TO DISK = 'D:\Backups\GradosTitulos_Diferencial.bak'
WITH
    DIFFERENTIAL,
    INIT,
    NAME = 'Respaldo Diferencial GradosTitulos',
    STATS = 10;
GO
```

Justificación técnica de la solución aplicada

El respaldo diferencial almacena únicamente los cambios realizados desde el último respaldo completo, reduciendo el tiempo de ejecución y el espacio de almacenamiento requerido.

Explicación de las buenas prácticas utilizadas en el proyecto

Se almacenó el respaldo en un archivo independiente y se utilizó la opción **STATS** para monitorear el progreso del proceso de copia de seguridad.

Ejercicio Propuesto 2: Respaldo periódico del Log de Transacciones (Transaction Log Backup)

Enunciado

Dado que la base de datos opera bajo el modelo de recuperación **FULL**, realiza un respaldo exclusivo del archivo de registro de transacciones (**LOG**) de **GradosTitulos** para liberar el espacio reutilizable dentro del archivo **.ldf** y asegurar la posibilidad de recuperar los datos hasta un minuto exacto antes de un fallo.

Script de la solución en T-SQL

```
USE master;
GO

-- Realizar un respaldo del Log de Transacciones
BACKUP LOG GradosTitulos
TO DISK = 'C:\Backups\GradosTitulos_Log.trn'
WITH
    INIT,
    NAME = 'Respaldo Log GradosTitulos',
    STATS = 10;
GO
```

Justificación técnica de la solución aplicada

El respaldo del log de transacciones permite conservar los registros de cambios realizados en la base de datos y facilita la recuperación de la información hasta un punto específico en el tiempo.

Explicación de las buenas prácticas utilizadas en el proyecto

Se realizó el respaldo del log en un archivo independiente para mantener una mejor organización de las copias de seguridad y facilitar su administración.

Proyecto 4: Implementación de roles y usuarios para seguridad en GradosTitulos

Enunciado

Crear un esquema de seguridad basado en roles integrados del sistema. Configura el login de servidor y usuario de base de datos **VendedorQhatu** (renombrado bajo el ecosistema institucional como **OperadorTramite**), asignándolo al rol **db_datawriter** para la gestión diaria del flujo administrativo. Asimismo, configura el usuario **ConsultaCliente** (renombrado como **AuditorSunedu**) vinculándolo exclusivamente al rol **db_datareader**.

Script de la solución en T-SQL

```
-- Crear el Login del servidor para OperadorTramite
CREATE LOGIN OperadorTramite
WITH PASSWORD = 'Operador123*';
GO

-- Crear el Login del servidor para AuditorSunedu
CREATE LOGIN AuditorSunedu
WITH PASSWORD = 'Auditor123*';
GO

USE GradosTitulos;
GO

-- Crear el usuario OperadorTramite
CREATE USER OperadorTramite
FOR LOGIN OperadorTramite;
GO

-- Asignar el usuario al rol db_datawriter
ALTER ROLE db_datawriter
ADD MEMBER OperadorTramite;
GO

-- Crear el usuario AuditorSunedu
CREATE USER AuditorSunedu
FOR LOGIN AuditorSunedu;
GO

-- Asignar el usuario al rol db_datareader
ALTER ROLE db_datareader
ADD MEMBER AuditorSunedu;
GO

-- Verificar los usuarios y roles asignados
SELECT
    dp.name AS Usuario,
    dr.name AS Rol
FROM sys.database_role_members drm
INNER JOIN sys.database_principals dp
    ON drm.member_principal_id = dp.principal_id
INNER JOIN sys.database_principals dr
    ON drm.role_principal_id = dr.principal_id
WHERE dp.name IN ('OperadorTramite', 'AuditorSunedu');
GO
```

Justificación técnica de la solución aplicada

Se crearon usuarios con permisos específicos según sus funciones, asignándolos a los roles integrados de SQL Server para controlar el acceso a la información de la base de datos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se aplicó el principio de mínimo privilegio, otorgando únicamente los permisos necesarios a cada usuario y verificando posteriormente la correcta asignación de los roles.

Ejercicio Propuesto 1: Creación de un Rol de Base de Datos personalizado (Custom Database Role)

Enunciado

El equipo de secretaría académica necesita un perfil de seguridad intermedio. Crea un rol personalizado dentro de la base de datos llamado **RolSecretariaAcademica**. Concédeles permisos de lectura (**SELECT**) en el esquema **Academico** y en el esquema **Tramite**, pero otórgales también permisos para insertar registros (**INSERT**) únicamente en la tabla **Tramite.Tramite**.

Script de la solución en T-SQL

```
USE GradosTitulos;
GO

-- Crear el rol personalizado
CREATE ROLE RolSecretariaAcademica;
GO

-- Otorgar permisos de lectura sobre los esquemas
GRANT SELECT
ON SCHEMA::Academico
TO RolSecretariaAcademica;
GO

GRANT SELECT
ON SCHEMA::Tramite
TO RolSecretariaAcademica;
GO

-- Otorgar permiso de inserción únicamente sobre la tabla Tramite
GRANT INSERT
ON Tramite.Tramite
TO RolSecretariaAcademica;
GO

-- Verificar los permisos del rol
SELECT
    dp.name AS Rol,
    pe.permission_name AS Permiso,
    pe.state_desc AS Estado
FROM sys.database_permissions pe
INNER JOIN sys.database_principals dp
ON pe.grantee_principal_id = dp.principal_id
WHERE dp.name = 'RolSecretariaAcademica';
GO
```

Justificación técnica de la solución aplicada

Se creó un rol personalizado para asignar permisos específicos al personal de secretaría académica, facilitando una administración más organizada de la seguridad.

Explicación de las buenas prácticas utilizadas en el proyecto

Los permisos fueron otorgados únicamente sobre los objetos requeridos, aplicando el principio de mínimo privilegio para mejorar la seguridad de la base de datos.

Ejercicio Propuesto 2: Denegación explícita de acceso mediante DENY

Enunciado

Para proteger la información financiera de la universidad, aplica una restricción estricta de seguridad bloqueando explícitamente al usuario **OperadorTramite** cualquier lectura o

visualización de datos de la tabla **Tramite.Pago**, prevaleciendo sobre cualquier otro rol que tenga asignado.

Script de la solución en T-SQL

```
USE GradosTitulos;
GO

-- Denegar permisos de lectura sobre la tabla Pago
DENY SELECT
ON Tramite.Pago
TO OperadorTramite;
GO

-- Verificar el permiso aplicado
SELECT
    dp.name AS Usuario,
    pe.permission_name AS Permiso,
    pe.state_desc AS Estado
FROM sys.database_permissions pe
INNER JOIN sys.database_principals dp
ON pe.grantee_principal_id = dp.principal_id
WHERE dp.name = 'OperadorTramite';
GO
```

Justificación técnica de la solución aplicada

La instrucción **DENY** bloquea explícitamente el acceso a la información de la tabla **Tramite.Pago**, garantizando una mayor protección de los datos financieros.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó **DENY** para restringir el acceso a información sensible y posteriormente se verificó que el permiso fuera aplicado correctamente al usuario.

Proyecto 5: Configuración granular de permisos en el módulo académico de GradosTitulos Enunciado

Aplica el principio de mínimo privilegio de forma granular. Concede al usuario **GerenteQhatu** (actualizado a **CoordinadorFacultad**) acceso exclusivo de solo lectura (**SELECT**) a la vista analítica **Evaluacion.V_NotaFinalSustentacion**. En paralelo, revoca explícitamente al rol o usuario **CajeroQhatu** (actualizado a **AsistenteTramite**) el permiso de modificación (**UPDATE**) sobre la tabla crítica **Tramite.Pago**.

Script de la solución en T-SQL

```

|-- Crear el login CoordinadorFacultad
CREATE LOGIN CoordinadorFacultad
WITH PASSWORD = 'Coordinador123*';
GO

-- Crear el login AsistenteTramite
CREATE LOGIN AsistenteTramite
WITH PASSWORD = 'Asistente123*';
GO

USE GradosTitulos;
GO

-- Crear el usuario CoordinadorFacultad
CREATE USER CoordinadorFacultad
FOR LOGIN CoordinadorFacultad;
GO

-- Crear el usuario AsistenteTramite
CREATE USER AsistenteTramite
FOR LOGIN AsistenteTramite;
GO

-- Conceder permiso SELECT sobre la vista
GRANT SELECT
ON Evaluacion.V_NotaFinalSustentacion
TO CoordinadorFacultad;
GO

-- Revocar el permiso UPDATE sobre la tabla Pago
REVOKE UPDATE
ON Tramite.Pago
FROM AsistenteTramite;
GO

-- Verificar los permisos otorgados
SELECT
    dp.name AS Usuario,
    pe.permission_name AS Permiso,
    pe.state_desc AS Estado
FROM sys.database_permissions pe
INNER JOIN sys.database_principals dp
ON pe.grantee_principal_id = dp.principal_id
WHERE dp.name IN ('CoordinadorFacultad', 'AsistenteTramite');
GO

```

Justificación técnica de la solución aplicada

Se otorgó acceso únicamente a los recursos necesarios para cada usuario, garantizando un control más seguro sobre la información académica y administrativa.

Explicación de las buenas prácticas utilizadas en el proyecto

Se aplicó el principio de mínimo privilegio mediante **GRANT** y **REVOKE**, verificando posteriormente que los permisos fueran asignados correctamente.

Ejercicio Propuesto 1: Permisos granulares a nivel de columnas (Column-Level Permissions)

Enunciado

El equipo de desarrollo técnico necesita realizar estadísticas de contacto con los docentes, pero por normativas de privacidad no deben ver información sensible. Concede al usuario **PracticanteSistemas** acceso de lectura (**SELECT**) únicamente a las columnas **IdPersona**, **Nombres**, **ApellidoPaterno** y **Correo** de la tabla **Academico.Persona**.

Script de la solución en T-SQL

```

USE master;
GO

-- Crear el login PracticanteSistemas
CREATE LOGIN PracticanteSistemas
WITH PASSWORD = 'Practicante123*';
GO

USE GradosTitulos;
GO

-- Crear el usuario PracticanteSistemas
CREATE USER PracticanteSistemas
FOR LOGIN PracticanteSistemas;
GO

-- Conceder permiso SELECT únicamente sobre columnas específicas
GRANT SELECT
(
    IdPersona,
    Nombres,
    ApellidoPaterno,
    Correo
)
ON Academico.Persona
TO PracticanteSistemas;
GO

-- Verificar los permisos del usuario
SELECT
    dp.name AS Usuario,
    pe.permission_name AS Permiso,
    pe.state_desc AS Estado
FROM sys.database_permissions pe
INNER JOIN sys.database_principals dp
ON pe.grantee_principal_id = dp.principal_id
WHERE dp.name = 'PracticanteSistemas';
GO

```

Justificación técnica de la solución aplicada

Se otorgó acceso únicamente a las columnas necesarias, protegiendo la información sensible de la tabla Academico.Persona.

Explicación de las buenas prácticas utilizadas en el proyecto

Se aplicaron permisos a nivel de columnas para cumplir el principio de mínimo privilegio y proteger la privacidad de los datos.

Ejercicio Propuesto 2: Concesión del permiso EXECUTE sobre Procedimientos Almacenados

Enunciado

Concede al usuario **OperadorTramite** permisos exclusivos para ejecutar el procedimiento almacenado **Tramite.SP_RegistrarTramite**, garantizando que pueda procesar nuevas solicitudes sin necesidad de darle accesos directos de escritura sobre las tablas involucradas.

Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- Conceder permiso para ejecutar el procedimiento almacenado
GRANT EXECUTE
ON OBJECT::Tramite.SP_RegistrarTramite
TO OperadorTramite;
GO

-- Verificar el permiso otorgado
SELECT
    dp.name AS Usuario,
    pe.permission_name AS Permiso,
    pe.state_desc AS Estado
FROM sys.database_permissions pe
INNER JOIN sys.database_principals dp
ON pe.grantee_principal_id = dp.principal_id
WHERE dp.name = 'OperadorTramite';
GO

```

Justificación técnica de la solución aplicada

Se concedió el permiso EXECUTE para permitir la ejecución del procedimiento almacenado sin otorgar acceso directo a las tablas de la base de datos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se asignó únicamente el permiso necesario para ejecutar el procedimiento, manteniendo un mayor nivel de seguridad sobre la información almacenada.

Proyecto 6: Identificación y solución de procesos lentos y bloqueos en GradosTítulos

Enunciado

Durante periodos masivos de carga académica del flujo de egresados, el servidor experimenta lentitud. Describe los indicadores clave de rendimiento a monitorizar mediante la interfaz gráfica del **Activity Monitor**. Simultáneamente, desarrolla una consulta avanzada en T-SQL utilizando vistas de administración dinámica (DMVs) para rastrear bloqueos de recursos en tiempo real en la base de datos

Script de la solución en T-SQL

```
USE GradosTítulos;
GO

-- Consultar bloqueos activos mediante DMVs
SELECT
    r.session_id AS Sesion,
    r.status AS Estado,
    r.blocking_session_id AS SesionBloqueadora,
    r.wait_type AS TipoEspera,
    r.wait_time AS TiempoEspera_ms,
    DB_NAME(r.database_id) AS BaseDatos,
    t.text AS ConsultaSQL
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.database_id = DB_ID('GradosTítulos')
AND r.blocking_session_id <> 0;
GO
```

Justificación técnica de la solución aplicada

Las vistas de administración dinámica (DMVs) permiten identificar bloqueos y sesiones activas en tiempo real, facilitando el análisis del rendimiento de la base de datos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizaron las DMVs del sistema para monitorear el estado del servidor sin afectar el funcionamiento normal de la base de datos y detectar oportunamente posibles bloqueos.

Ejercicio Propuesto 1: Identificación de consultas que consumen mayor tiempo de CPU

Enunciado

Escribe una consulta en T-SQL que analice los planes de ejecución que se encuentran almacenados en la memoria caché del servidor. El script debe listar las **5 consultas** que han consumido la mayor cantidad de tiempo de procesador (CPU) en la base de datos **GradosTítulos**.

Script de la solución en T-SQL

```
USE GradosTitulos;
GO

-- Consultar las 5 consultas con mayor consumo de CPU
SELECT TOP (5)
    qs.total_worker_time AS TiempoCPU,
    qs.execution_count AS Ejecuciones,
    qs.total_worker_time / qs.execution_count AS PromedioCPU,
    DB_NAME(st.dbid) AS BaseDatos,
    st.text AS ConsultaSQL
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
WHERE DB_NAME(st.dbid) = 'GradosTitulos'
ORDER BY qs.total_worker_time DESC;
GO
```

Justificación técnica de la solución aplicada

La consulta identifica las instrucciones que consumen mayor tiempo de CPU, facilitando la detección de consultas que requieren optimización.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizaron las DMVs del sistema para analizar el rendimiento de las consultas sin afectar el funcionamiento de la base de datos.

Ejercicio Propuesto 2: Rastreo del uso de los Índices en las consultas (Index Usage Stats) Enunciado

Implementa una consulta analítica en T-SQL para verificar el uso real de los índices no agrupados creados en los esquemas de **GradosTitulos**, identificando si hay índices que no están siendo utilizados en las búsquedas pero que siguen consumiendo espacio en disco.

Script de la solución en T-SQL

```
USE GradosTitulos;
GO

-- Consultar el uso de los índices
SELECT
    OBJECT_NAME(i.object_id) AS Tabla,
    i.name AS Indice,
    i.type_desc AS TipoIndice,
    ISNULL(s.user_seeks, 0) AS Busquedas,
    ISNULL(s.user_scans, 0) AS Escaneos,
    ISNULL(s.user_lookups, 0) AS Consultas,
    ISNULL(s.user_updates, 0) AS Actualizaciones
FROM sys.indexes i
LEFT JOIN sys.dm_db_index_usage_stats s
ON i.object_id = s.object_id
AND i.index_id = s.index_id
AND s.database_id = DB_ID('GradosTitulos')
WHERE i.type_desc = 'NONCLUSTERED'
ORDER BY Busquedas ASC;
GO
```

Justificación técnica de la solución aplicada

La consulta permite identificar los índices que tienen poco o ningún uso, facilitando su evaluación para mejorar el rendimiento y optimizar el almacenamiento.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizaron las estadísticas de uso de índices proporcionadas por SQL Server para obtener información confiable y apoyar las tareas de optimización de la base de datos.

Proyecto 7: Automatización de respaldos y limpieza periódica del sistema de Grados

Enunciado

Diseña los scripts transaccionales necesarios para automatizar tareas rutinarias a través de **SQL Server Agent**. El primer proceso ejecutará un respaldo completo programado diariamente de la base de datos. El segundo proceso se encargará de purgar semanalmente todos aquellos registros de auditoría almacenados en la tabla de observaciones que superen los **15 días** de antigüedad y cuyo estado ya se encuentre resuelto.

Script de la solución en T-SQL

```
USE msdb;
GO

-- Crear el Job para el respaldo diario
EXEC sp_add_job
    @job_name = 'Backup Diario GradosTitulos';
GO

EXEC sp_add_jobstep
    @job_name = 'Backup Diario GradosTitulos',
    @step_name = 'Respaldo Completo',
    @subsystem = 'TSQL',
    @command = '
        BACKUP DATABASE GradosTitulos
        TO DISK = ''D:\Backups\GradosTitulos_Diario.bak''
        WITH INIT, STATS = 10;
    ';
GO

-- Crear el Job para la limpieza semanal
EXEC sp_add_job
    @job_name = 'Limpieza Observaciones';
GO

EXEC sp_add_jobstep
    @job_name = 'Limpieza Observaciones',
    @step_name = 'Eliminar Observaciones Antiguas',
    @subsystem = 'TSQL',
    @command = '
        DELETE FROM Tramite.Observaciones
        WHERE FechaRegistro < DATEADD(DAY,-15,GETDATE())
        AND Estado = ''Resuelto'';
    ';
GO
```

Justificación técnica de la solución aplicada

La automatización mediante **SQL Server Agent** permite ejecutar respaldos y tareas de mantenimiento de forma programada, reduciendo la intervención manual y mejorando la disponibilidad de la base de datos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se separaron las tareas en dos **Jobs** independientes para facilitar su administración y mantenimiento, automatizando tanto los respaldos como la limpieza periódica de registros.

Ejercicio Propuesto 1: Notificación de alertas ante fallos en los Jobs del Agente

Enunciado

Escribe el script transaccional para configurar un operador del sistema llamado **DBA_Alertas** en el Agente de SQL Server, de tal manera que el servidor envíe correos electrónicos automáticos si el **Job** de respaldo diario de la universidad falla.

Script de la solución en T-SQL

```
USE msdb;
GO

-- Crear el operador del sistema
EXEC sp_add_operator
    @name = 'DBA_Alertas',
    @enabled = 1,
    @email_address = 'dba@universidad.edu.pe';
GO

-- Configurar la notificación del Job
EXEC sp_update_job
    @job_name = 'Backup Diario GradosTitulos',
    @notify_level_email = 2,
    @notify_email_operator_name = 'DBA_Alertas';
GO
```

Justificación técnica de la solución aplicada

La configuración de un operador permite recibir notificaciones automáticas cuando un Job presenta fallos, facilitando una respuesta oportuna ante incidencias.

Explicación de las buenas prácticas utilizadas en el proyecto

Se configuró un operador exclusivo para recibir alertas y se asoció únicamente al Job de respaldo diario para mejorar el monitoreo de las tareas programadas.

Ejercicio Propuesto 2: Automatización de la Reconstrucción de Índices Fragmentados

Enunciado

Desarrolla un script de mantenimiento automatizado para evaluar los índices de la base de datos GradosTitulos y ejecutar una reconstrucción completa (ALTER INDEX REBUILD) en aquellos índices que superen un 30% de fragmentación física, garantizando la velocidad de las consultas.

Script de la solución en T-SQL

```
USE GradosTitulos;
GO

DECLARE @Tabla NVARCHAR(200);
DECLARE @Indice NVARCHAR(200);
DECLARE @SQL NVARCHAR(MAX);

DECLARE Indices CURSOR FOR
SELECT
    OBJECT_NAME(ps.object_id),
    i.name
FROM sys.dm_db_index_physical_stats
(
    DB_ID('GradosTitulos'),
    NULL,
    NULL,
    NULL,
    'LIMITED'
) ps
INNER JOIN sys.indexes i
ON ps.object_id = i.object_id
AND ps.index_id = i.index_id
WHERE ps.avg_fragmentation_in_percent > 30
AND i.type_desc = 'NONCLUSTERED';

OPEN Indices;

FETCH NEXT FROM Indices INTO @Tabla, @Indice;

WHILE @@FETCH_STATUS = 0
BEGIN
    SET @SQL = 'ALTER INDEX [' + @Indice + '] ON [' + @Tabla + '] REBUILD';
    EXEC(@SQL);

    FETCH NEXT FROM Indices INTO @Tabla, @Indice;
END

CLOSE Indices;
DEALLOCATE Indices;
GO
```

Justificación técnica de la solución aplicada

La reconstrucción de índices con alta fragmentación mejora el rendimiento de las consultas y optimiza el acceso a los datos almacenados.

Explicación de las buenas prácticas utilizadas en el proyecto

Se reconstruyeron únicamente los índices con una fragmentación superior al **30%**, optimizando el rendimiento sin realizar mantenimiento innecesario.

Proyecto 8: Reestructuración física y alteración de esquemas en tablas de Pedidos

Enunciado

Con el fin de expandir el control analítico del proceso de sustentaciones y registros, adapta el ejercicio agregando de manera segura una columna denominada **PrioridadAtencion** de tipo entero (INT) a la estructura de la tabla **Tramite.Tramite**. Posteriormente, y emulando la corrección de requerimientos del sistema, elimina por completo la columna redundante **Observaciones** de dicha tabla.

Script de la solución en T-SQL

```
USE GradosTitulos;
GO

-- Agregar la columna PrioridadAtencion
ALTER TABLE Tramite.Tramite
ADD PrioridadAtencion INT;
GO

-- Eliminar la columna Observaciones
ALTER TABLE Tramite.Tramite
DROP COLUMN Observaciones;
GO

-- Verificar la estructura actual de la tabla
EXEC sp_help 'Tramite.Tramite';
GO
```

Justificación técnica de la solución aplicada

La modificación de la estructura de la tabla permite incorporar nuevos requerimientos del sistema y eliminar columnas que ya no son necesarias, manteniendo el modelo de datos actualizado.

Explicación de las buenas prácticas utilizadas en el proyecto

Se realizaron las modificaciones mediante ALTER TABLE y posteriormente se verificó la estructura de la tabla para confirmar que los cambios fueron aplicados correctamente.

Ejercicio Propuesto 1: Incorporación de Restricciones CHECK complejas post-creación

Enunciado

Modifica la tabla Tramite. Pago agregando una nueva restricción de comprobación (CHECK CONSTRAINT) para asegurar que ningún registro acepte combinaciones donde el campo **Monto** sea menor o igual a cero, obligando a que todo pago ingresado al sistema sea siempre una cantidad positiva.

Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- Agregar la restricción CHECK
ALTER TABLE Tramite.Pago
ADD CONSTRAINT CK_Pago_MontoPositivo
CHECK (Monto > 0);
GO

-- Verificar la restricción creada
SELECT
    name AS Restriccion,
    type_desc AS Tipo
FROM sys.objects
WHERE name = 'CK_Pago_MontoPositivo';
GO

```

Justificación técnica de la solución aplicada

La restricción **CHECK** garantiza que el campo **Monto** solo acepte valores positivos, preservando la integridad de los datos registrados.

Explicación de las buenas prácticas utilizadas en el proyecto

Se implementó una restricción a nivel de base de datos para validar automáticamente la información antes de almacenarla.

Ejercicio Propuesto 2: Modificación del tipo de datos de una columna con validación de nulidad

Enunciado

Por requerimientos de ampliación de cobertura de texto, modifica la columna **NroRecibo** de la tabla **Tramite.Pago**, elevando su tipo de datos de **VARCHAR(30)** a **VARCHAR(60)** y asegurando que mantenga su propiedad de permitir valores nulos (**NULL**).

Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- Modificar el tipo de dato de la columna
ALTER TABLE Tramite.Pago
ALTER COLUMN NroRecibo VARCHAR(60) NULL;
GO

-- Verificar la modificación realizada
SELECT
    COLUMN_NAME,
    DATA_TYPE,
    CHARACTER_MAXIMUM_LENGTH,
    IS_NULLABLE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'Tramite'
AND TABLE_NAME = 'Pago'
AND COLUMN_NAME = 'NroRecibo';
GO

```

Justificación técnica de la solución aplicada

La modificación del tipo de dato permite almacenar un mayor número de caracteres sin afectar la propiedad de aceptar valores nulos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se conservó la propiedad **NULL** durante la modificación de la columna y se verificó posteriormente que la estructura se actualizó correctamente.

Proyecto 9: Implementación de auditoría forense automatizada mediante Triggers

Enunciado

Es necesario auditar por políticas de protección de datos personales toda eliminación física sobre el padrón de personas. Diseña e implementa una tabla de historial denominada **Academico.AuditoriaPersona**. Acto seguido, desarrolla un desencadenador (**Trigger**) para el evento **DELETE** en la tabla **Academico.Persona**, el cual capturará los datos eliminados, el usuario del sistema que ejecutó la acción y el momento exacto del incidente.

Script de la solución en T-SQL

```
USE GradosTitulos;
GO

-- Crear la tabla de auditoría
CREATE TABLE Academico.AuditoriaPersona
(
    IdAuditoria INT IDENTITY(1,1) PRIMARY KEY,
    IdPersona INT,
    Nombres VARCHAR(100),
    ApellidoPaterno VARCHAR(100),
    ApellidoMaterno VARCHAR(100),
    UsuarioSistema SYSNAME,
    FechaEliminacion DATETIME
);
GO

-- Crear el Trigger para auditar eliminaciones
CREATE TRIGGER TR_Auditoria_EliminarPersona
ON Academico.Persona
AFTER DELETE
AS
BEGIN
    INSERT INTO Academico.AuditoriaPersona
    (
        IdPersona,
        Nombres,
        ApellidoPaterno,
        ApellidoMaterno,
        UsuarioSistema,
        FechaEliminacion
    )
    SELECT
        IdPersona,
        Nombres,
        ApellidoPaterno,
        ApellidoMaterno,
        SYSTEM_USER,
        GETDATE()
    FROM deleted;
END;
GO

-- Verificar la existencia del Trigger
SELECT
    name AS NombreTrigger,
    type_desc AS Tipo
FROM sys.triggers
WHERE name = 'TR_Auditoria_EliminarPersona';
GO
```

Justificación técnica de la solución aplicada

La tabla de auditoría y el trigger permiten registrar automáticamente las eliminaciones realizadas sobre la tabla Académico. Persona, facilitando el seguimiento de los cambios y el control de la información.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó un trigger AFTER DELETE para registrar cada eliminación sin intervención manual y se verificó su correcta creación mediante las vistas del sistema.

Ejercicio Propuesto 1: Trigger de Auditoría para el control de Modificaciones (UPDATE) Enunciado

Desarrolla un trigger llamado **TR_Auditoria_ModificarPago** en la tabla **Tramite.Pago**. Este trigger debe activarse tras cualquier modificación de registros (**AFTER UPDATE**), guardando en una tabla de auditoría el valor anterior del monto cobrado, el nuevo valor ingresado y el usuario responsable del cambio.

Script de la solución en T-SQL

```
USE GradosTitulos;
GO

-- Crear la tabla de auditoría para modificaciones
CREATE TABLE Tramite.AuditoriaPago
(
    IdAuditoria INT IDENTITY(1,1) PRIMARY KEY,
    IdPago INT,
    MontoAnterior DECIMAL(10,2),
    MontoNuevo DECIMAL(10,2),
    UsuarioSistema SYSNAME,
    FechaModificacion DATETIME
);
GO

-- Crear el Trigger de auditoría
CREATE TRIGGER TR_Auditoria_ModificarPago
ON Tramite.Pago
AFTER UPDATE
AS
BEGIN
    INSERT INTO Tramite.AuditoriaPago
    (
        IdPago,
        MontoAnterior,
        MontoNuevo,
        UsuarioSistema,
        FechaModificacion
    )
    SELECT
        d.IdPago,
        d.Monto,
        i.Monto,
        SYSTEM_USER,
        GETDATE()
    FROM deleted d
    INNER JOIN inserted i
        ON d.IdPago = i.IdPago;
END;
GO

-- Verificar la existencia del Trigger
SELECT
    name AS NombreTrigger,
    type_desc AS Tipo
FROM sys.triggers
WHERE name = 'TR_Auditoria_ModificarPago';
GO
```

Justificación técnica de la solución aplicada

El trigger registra automáticamente las modificaciones realizadas sobre el monto de los pagos, permitiendo mantener un historial de los cambios efectuados.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó un trigger **AFTER UPDATE** para registrar únicamente los cambios realizados y se verificó su correcta creación mediante las vistas del sistema.

Ejercicio Propuesto 2: Trigger de control de seguridad institucional (Restricción de Horarios)

Enunciado

Implementa un trigger de seguridad de tipo **INSTEAD OF** en la tabla **Academico.Docente** para bloquear cualquier intento de eliminación o modificación de registros los fines de semana (sábados y domingos), cancelando la transacción automáticamente por motivos de seguridad informática.

Script de la solución en T-SQL

```
USE GradosTitulos;
GO

-- Crear el Trigger de seguridad
CREATE TRIGGER TR_ControlHorarioDocente
ON Academico.Docente
INSTEAD OF UPDATE, DELETE
AS
BEGIN
    IF DATENAME(WEEKDAY, GETDATE()) IN ('Saturday', 'Sunday')
    BEGIN
        RAISERROR('No se permiten modificaciones ni eliminaciones de docentes durante los fines de semana.',16,1);
        ROLLBACK TRANSACTION;
        RETURN;
    END
END;
GO

-- Verificar la existencia del Trigger
SELECT
    name AS NombreTrigger,
    type_desc AS Tipo
FROM sys.triggers
WHERE name = 'TR_ControlHorarioDocente';
GO
```

Justificación técnica de la solución aplicada

El trigger restringe las operaciones de modificación y eliminación durante los fines de semana, fortaleciendo la seguridad y el control sobre la información institucional.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó un trigger INSTEAD OF para impedir la ejecución de operaciones no autorizadas y garantizar el cumplimiento de las políticas de seguridad establecidas.

Proyecto 10: Simulación de recuperación y restauración completa ante contingencias de datos

Enunciado

Simula un escenario catastrófico provocado por error humano mediante el borrado accidental de datos críticos en el catálogo institucional. Posteriormente, ejecuta la restauración completa de la base de datos **GradosTitulos** utilizando el respaldo de seguridad (**.bak**) generado previamente en el Proyecto 3. Finaliza validando la correcta restitución y consistencia de los datos mediante una consulta de verificación.

Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- Simular la eliminación accidental de datos
DELETE FROM Catalogo.EstadoTramite;
GO

USE master;
GO

-- Restaurar la base de datos desde el respaldo completo
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Full.bak'
WITH REPLACE, RECOVERY;
GO

-- Verificar la recuperación de los datos
USE GradosTitulos;
GO

SELECT *
FROM Catalogo.EstadoTramite;
GO

```

Justificación técnica de la solución aplicada

La restauración de la base de datos desde un respaldo completo permite recuperar la información eliminada y restablecer el estado operativo de la base de datos ante una contingencia.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó un respaldo previamente generado para realizar la recuperación y posteriormente se verificó la correcta restitución de los datos mediante una consulta.

Ejercicio Propuesto 1: Restauración parcial de tablas mediante exportación e inserción controlada

Enunciado

Ante un error operativo, se borraron accidentalmente los datos de la tabla de parámetros Catalogo.EstadoTramite. En lugar de restaurar toda la base de datos y perder las transacciones del día, utiliza comandos avanzados de consulta entre bases de datos (Cross-Database Queries) para recuperar la información desde una copia de respaldo montada en el mismo servidor bajo el nombre GradosTitulos_Auxiliar.

Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- Restaurar los datos desde la base de datos auxiliar
INSERT INTO Catalogo.EstadoTramite
SELECT *
FROM GradosTitulos_Auxiliar.Catalogo.EstadoTramite;
GO

-- Verificar la recuperación de los registros
SELECT *
FROM Catalogo.EstadoTramite;
GO

```

Justificación técnica de la solución aplicada

La recuperación mediante consultas entre bases de datos permite restaurar únicamente la información necesaria, evitando reemplazar toda la base de datos y conservando las transacciones recientes.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó una base de datos auxiliar como fuente de recuperación y se verificó posteriormente que los registros fueron restaurados correctamente.

Ejercicio Propuesto 2: Verificación de la consistencia física con DBCC CHECKDB

Enunciado

Tras completar una restauración de emergencia en el servidor, ejecuta el comando de diagnóstico avanzado DBCC CHECKDB para realizar una auditoría completa de la integridad estructural y lógica de las páginas de datos e índices de GradosTitulos, asegurando que no existan errores ocultos en el almacenamiento.

Script de la solución en T-SQL

```
USE master;  
GO  
  
-- Verificar la integridad de la base de datos  
DBCC CHECKDB ('GradosTitulos');  
GO
```

Justificación técnica de la solución aplicada

El comando DBCC CHECKDB permite comprobar la integridad física y lógica de la base de datos, detectando posibles errores en las páginas de datos e índices.

Explicación de las buenas prácticas utilizadas en el proyecto

Se ejecutó la verificación de integridad después del proceso de restauración para confirmar que la base de datos se encuentra en un estado consistente y libre de errores.